



MiniSocial - Complete Project Documentation



Status **Active**

Version **1.0.0**

License **MIT**

Last Updated **Feb 2026**

Author: [@devberatzengin](#) | **Repository:** [GitHub](#)



Quick Navigation

Section	Details
Overview	Project description & objectives
Architecture	System design & flow diagrams
Tech Stack	Technologies & dependencies
Screenshots	UI & working application
Installation	Complete setup guide
Database	Schema & relationships
API Docs	Endpoint reference
Authentication	JWT & security
Use Cases	Real-world examples
Structure	Codebase organization

Section	Details
 Development	Adding features guide
 Deployment	Production deployment
 Troubleshooting	Common issues & fixes
 Contributing	How to contribute

Project Overview

| A lightweight, modern social blogging platform built with cutting-edge Python technologies

MiniSocial is a lightweight social blogging platform that allows users to: -  Create and share blog posts with rich content -  Engage through comments and interactive discussions -  Like/unlike posts to show appreciation -  Follow/unfollow users for personalized feeds -  Search posts and discover users -  Secure accounts with JWT authentication -  Responsive design for all devices

Key Features at a Glance

Feature	Description	Status
 JWT Authentication	Secure token-based authentication	 Complete
 Post Management	Create, read, delete posts	 Complete
 Comments	Comment on posts	 Complete
 Likes System	Like/unlike posts & comments	 Complete
 Follow System	Follow/unfollow users	 Complete
 Advanced Search	Search posts & users	 Complete
 User Profiles	Public profiles with stats	 Complete
 Feed Algorithm	Personalized post feed	 Complete
 Dark Mode	Dark theme option	 Planned
 Image Upload	Post image attachments	 Planned
 Notifications	Real-time notifications	 Planned
 Direct Messages	User-to-user messaging	 Planned

 Key Objectives

Goal	Description
 User Experience	Provide simple, intuitive social networking
 Best Practices	Demonstrate modern web development patterns
 Technology	Showcase cutting-edge Python frameworks
 Security	Ensure data protection with JWT & hashing
 Scalability	Build architecture ready for growth
 Learning	Educational resource for developers

 Target Users

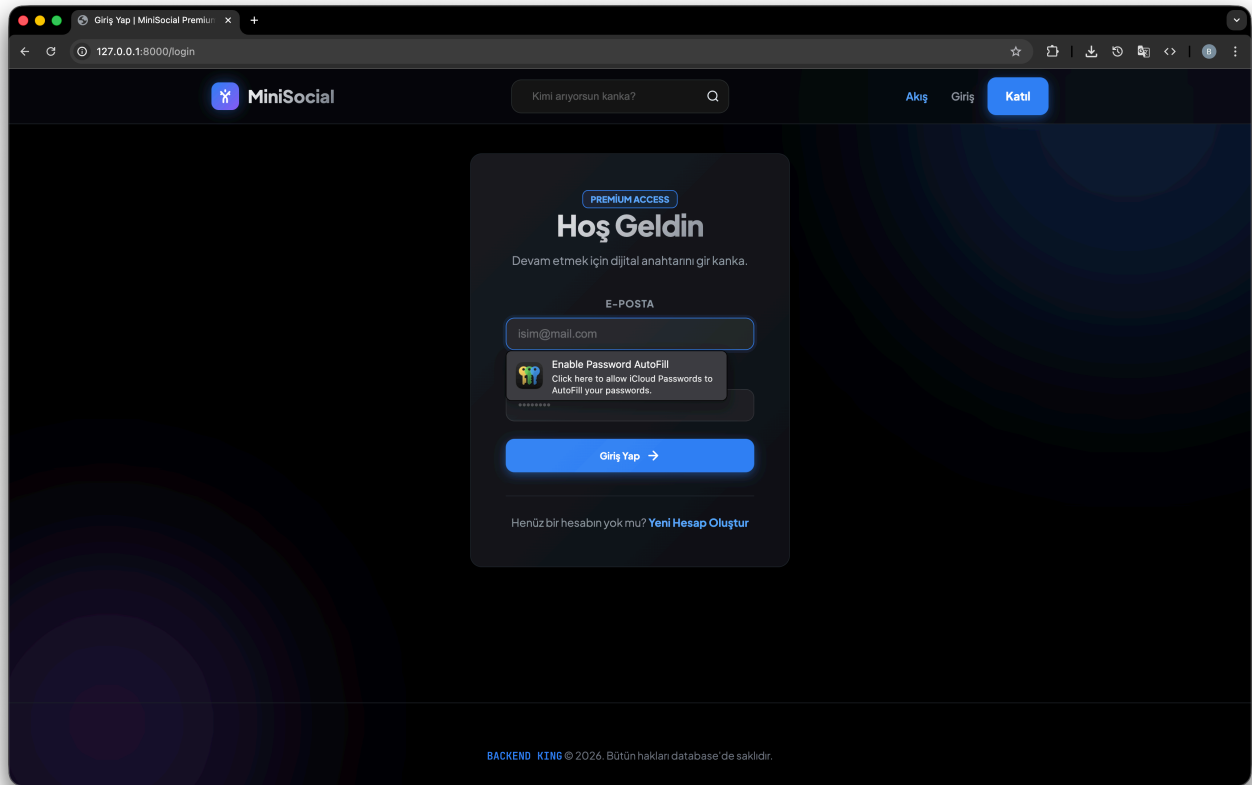
	Developers Learning FastAPI
	Content Creators
	Teams Building Social Features
	Students Studying Web Dev

 Screenshots & Live Demo

Note: Add your application screenshots here. Follow these steps: 1. Run your application: `cd app && uvicorn main:app --reload` 2. Take screenshots of each page: - Login page: `http://localhost:8000/login` - Registration: `http://localhost:8000/register` - Feed: `http://localhost:8000/feed` - Profile: `http://localhost:8000/profile` - Post detail: `http://localhost:8000/post/1` 3. Create screenshots/ folder in project root 4. Add images below with: `![Alt text](screenshots/image-name.png)`

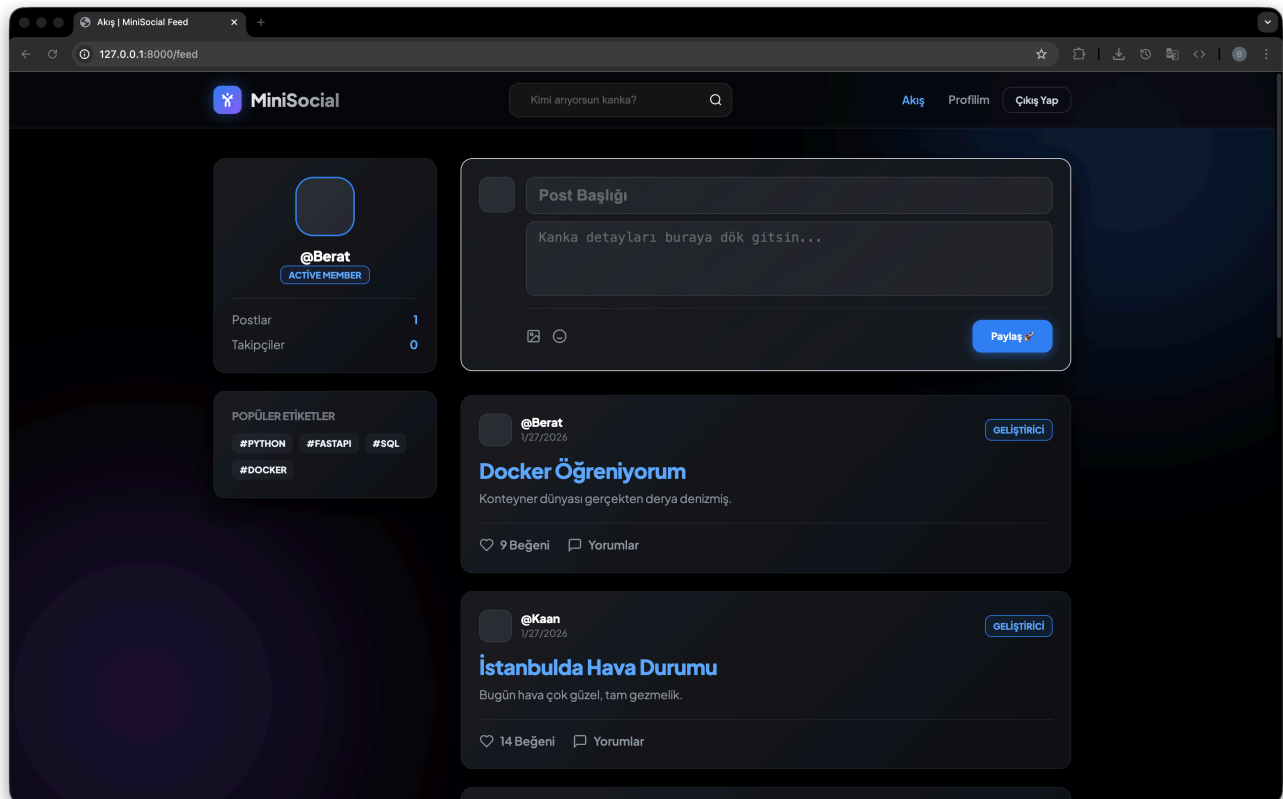
Application Pages

Login & Registration



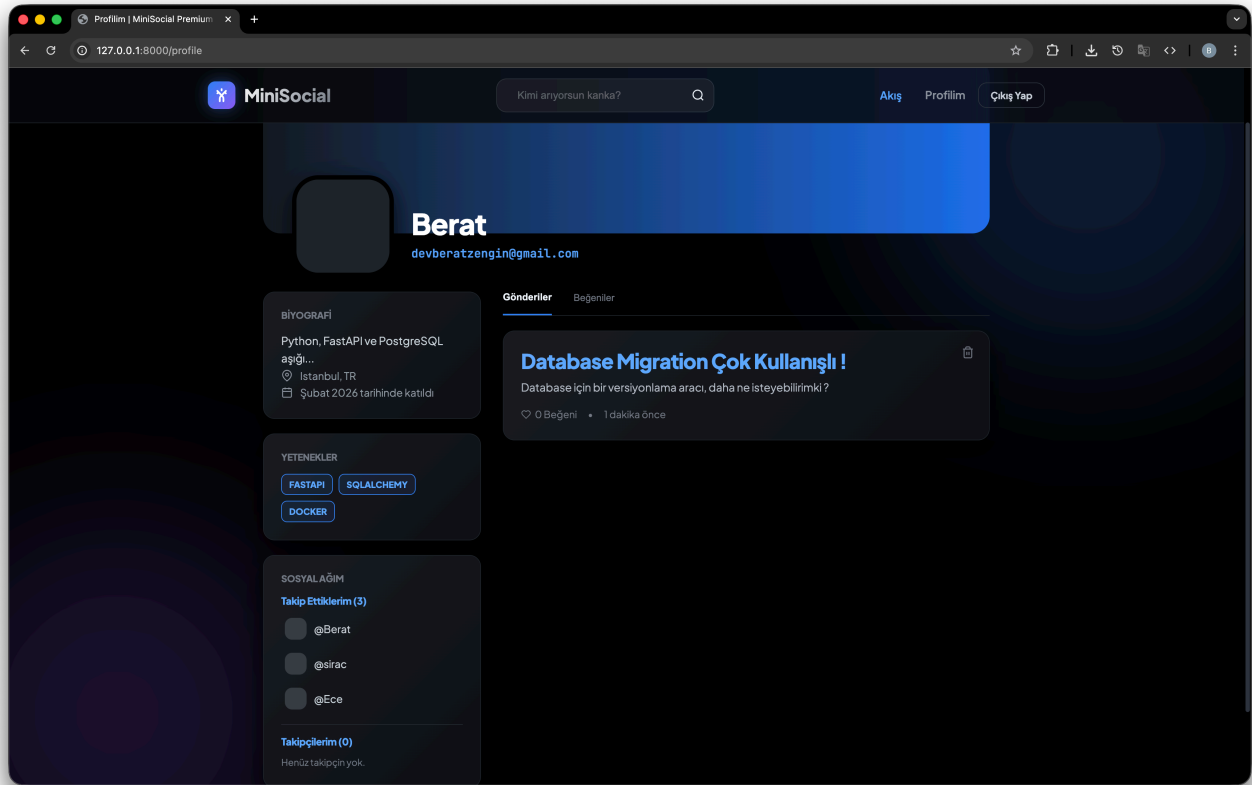
Screenshot: Login page — clean login form with username/password fields.

Feed Page



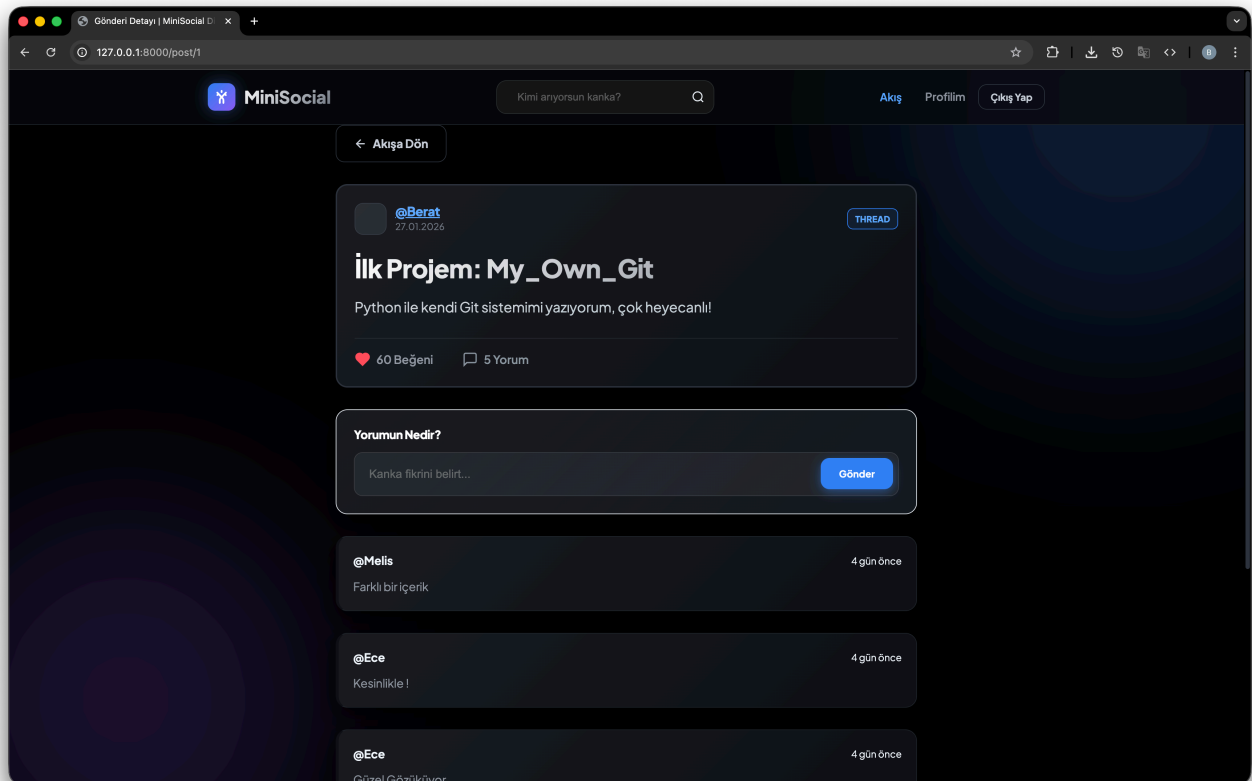
Screenshot: Feed — list of posts with interaction buttons (like/comment).

Profile Page



Screenshot: Profile — user info, bio, posts and follower stats.

Post Detail



Screenshot: Post detail — full post view with comments and actions.

API Documentation UI

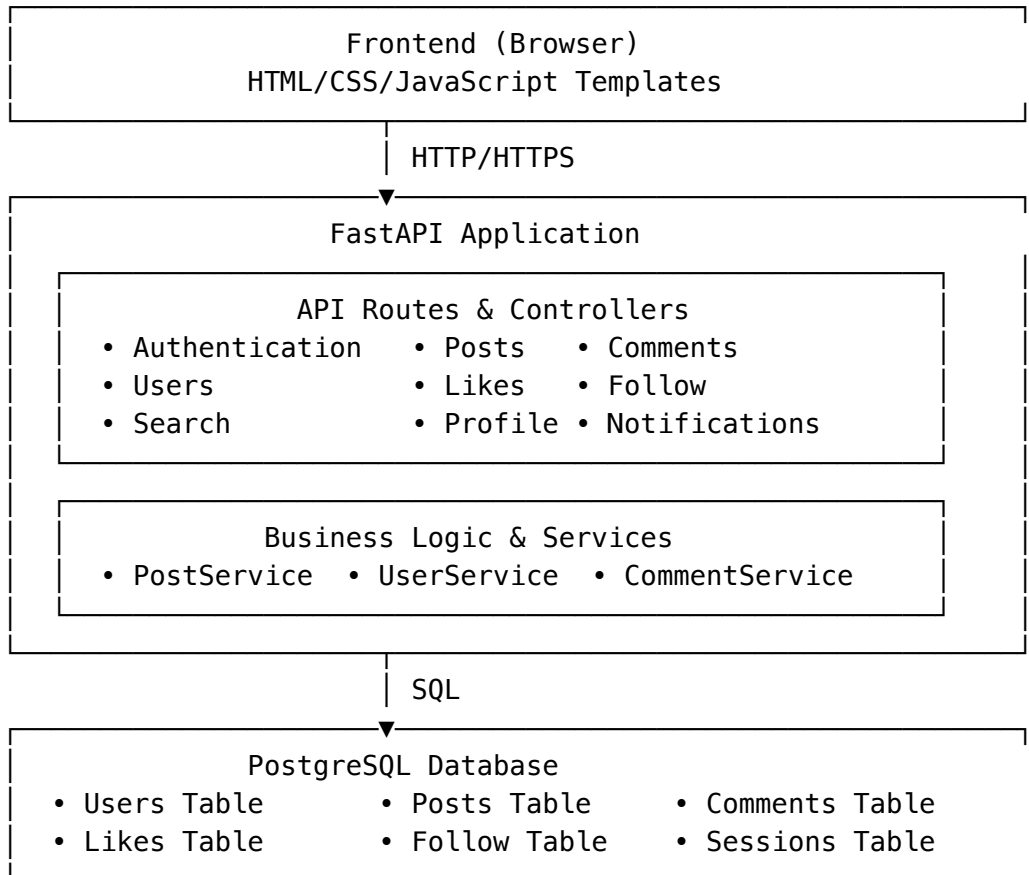
The application includes interactive API documentation:

- **Swagger UI** (Recommended): <http://localhost:8000/docs>
- **ReDoc** (Alternative): <http://localhost:8000/redoc>

Both provide: -  Full API endpoint listing -  Request/response schemas -  Try-it-out functionality -  Authentication testing

Architecture

High-Level Architecture



Request/Response Flow

1. **Client Request**: User sends HTTP request through browser or API client
2. **Authentication**: JWT token validated in request header
3. **Route Handler**: FastAPI route processes request
4. **Service Layer**: Business logic executed

5. **Database Query:** SQLAlchemy ORM queries PostgreSQL

6. **Response:** JSON response sent back to client

Technology Stack

Core Technologies

Backend



Database



Frontend



Security



Detailed Stack

Backend Framework - **FastAPI** (0.95+): Modern, fast async web framework with automatic API documentation - **Uvicorn**: Lightning-fast ASGI web server implementation - **Python** (3.8+): Core language with async/await support

Database & ORM - **PostgreSQL** (10+): Production-grade relational database - **SQLAlchemy** (2.0+): Powerful SQL toolkit and ORM - **Alembic** (1.10+): Lightweight database migration tool - **psycopg2-binary**: PostgreSQL driver for Python

Frontend & Templates - **Jinja2**: Powerful template engine for dynamic HTML - **HTML5**: Semantic markup for modern web - **CSS3**: Modern styling with responsive design

Security & Authentication - **JWT**: Stateless token-based authentication - **Passlib**: Password hashing with bcrypt - **python-dotenv**: Secure environment variable management

Development Tools - **pytest**: Comprehensive testing framework - **Black**: Code formatter for consistency - **flake8**: Style guide enforcement - **mypy**: Static type checking

Installation & Setup

Step 1: Prerequisites

System Requirements: - Python 3.8 or higher - PostgreSQL 10 or higher - Git - pip (Python package manager)

Check installed versions:

```
python --version
psql --version
git --version
```

Step 2: Clone Repository

```
git clone https://github.com/devberatzengin/Basic-Blog-App.git
cd "Basic Blog App"
```

Step 3: Create Virtual Environment

```
# Create venv
python -m venv venv

# Activate venv
# On macOS/Linux:
source venv/bin/activate

# On Windows:
venv\Scripts\activate
```

Step 4: Install Dependencies

```
pip install -r requirements.txt
```

What's installed: - fastapi: Web framework - uvicorn: Web server - sqlalchemy: ORM - psycopg2-binary: PostgreSQL driver - alembic: Migrations - python-dotenv: Environment management

Step 5: Setup Database

```
# Create database
createdb blog_db
```



```
# Alternative using psql:  
psql -U postgres -c "CREATE DATABASE blog_db;"
```

Step 6: Configure Environment

Create .env file in project root:

```
# Database Configuration  
DATABASE_URL=postgresql://username:password@localhost:5432/blog_db  
  
# Security  
SECRET_KEY=your-super-secret-key-here-change-this-in-production  
ALGORITHM=HS256  
ACCESS_TOKEN_EXPIRE_MINUTES=30  
  
# Server (Optional)  
DEBUG=True  
SERVER_HOST=0.0.0.0  
SERVER_PORT=8000
```

How to generate SECRET_KEY:

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Step 7: Run Database Migrations

```
cd app  
alembic upgrade head  
cd ..
```

This creates all necessary tables in the database.

Step 8: Start Server

```
cd app  
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Output:

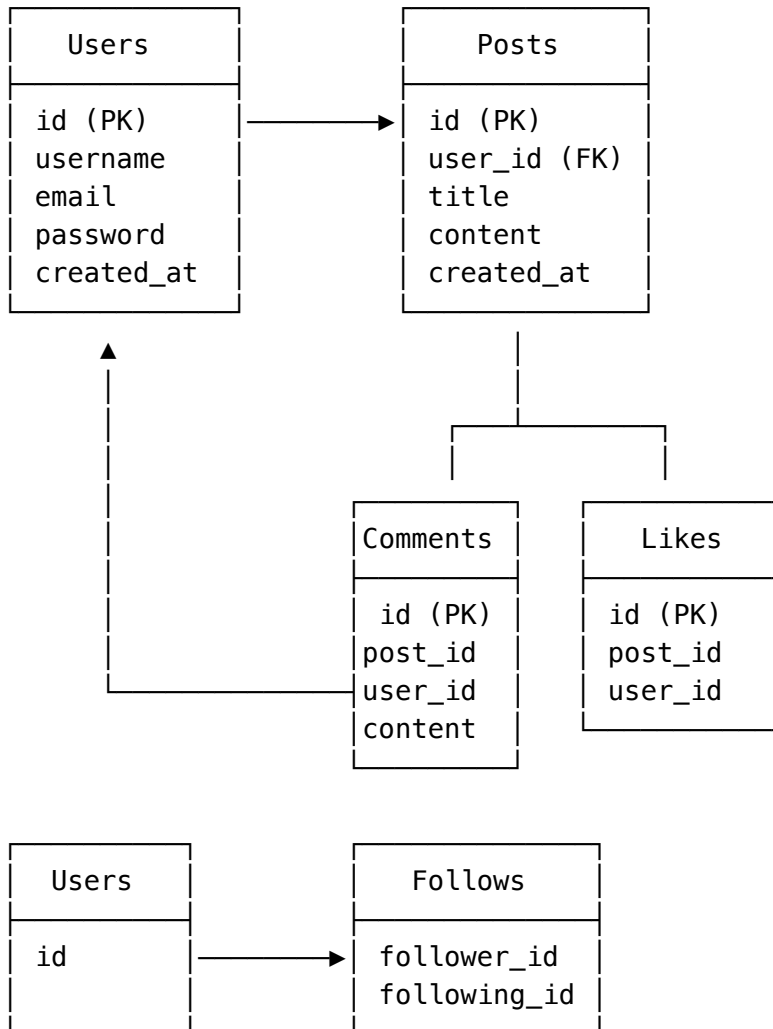
```
INFO:      Uvicorn running on http://0.0.0.0:8000  
INFO:      Application startup complete
```

Step 9: Access Application

- **Web:** <http://localhost:8000>
- **API Docs:** <http://localhost:8000/docs>
- **Alternative API Docs:** <http://localhost:8000/redoc>

Database Schema

ER Diagram



Tables Detailed

Users Table

```

CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(255) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(255),
  last_name VARCHAR(255),
  bio TEXT,
  profile_picture_url VARCHAR(500),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
  
```

Fields: - id: Unique user identifier - username: Unique username for login - email: User email address - password_hash: Hashed password (never store plain text!) - first_name, last_name: User's name - bio: User biography - profile_picture_url: Avatar image URL - created_at: Registration timestamp - updated_at: Last update timestamp

Posts Table

```
CREATE TABLE posts (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL,  
  title VARCHAR(500) NOT NULL,  
  content TEXT NOT NULL,  
  likes_count INTEGER DEFAULT 0,  
  comments_count INTEGER DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

Fields: - id: Unique post identifier - user_id: Author of the post - title: Post title - content: Post content/body - likes_count: Number of likes (cached) - comments_count: Number of comments (cached) - created_at: Post creation date - updated_at: Last edit date

Comments Table

```
CREATE TABLE comments (  
  id SERIAL PRIMARY KEY,  
  post_id INTEGER NOT NULL,  
  user_id INTEGER NOT NULL,  
  content TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

Likes Table

```
CREATE TABLE likes (  
  id SERIAL PRIMARY KEY,  
  post_id INTEGER NOT NULL,  
  user_id INTEGER NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE(post_id, user_id),  
  FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

Follow Table

```
CREATE TABLE follows (  
  id SERIAL PRIMARY KEY,
```

```
    follower_id INTEGER NOT NULL,  
    following_id INTEGER NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE(follower_id, following_id),  
    FOREIGN KEY (follower_id) REFERENCES users(id) ON DELETE CASCADE,  
    FOREIGN KEY (following_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

API Documentation

Base URL

`http://localhost:8000/api`

Authentication Header

For protected endpoints, include:

Authorization: Bearer <your_jwt_token>

Response Format

All responses are JSON:

```
{  
  "status": "success",  
  "data": {},  
  "message": "Operation successful"  
}
```

Error Responses

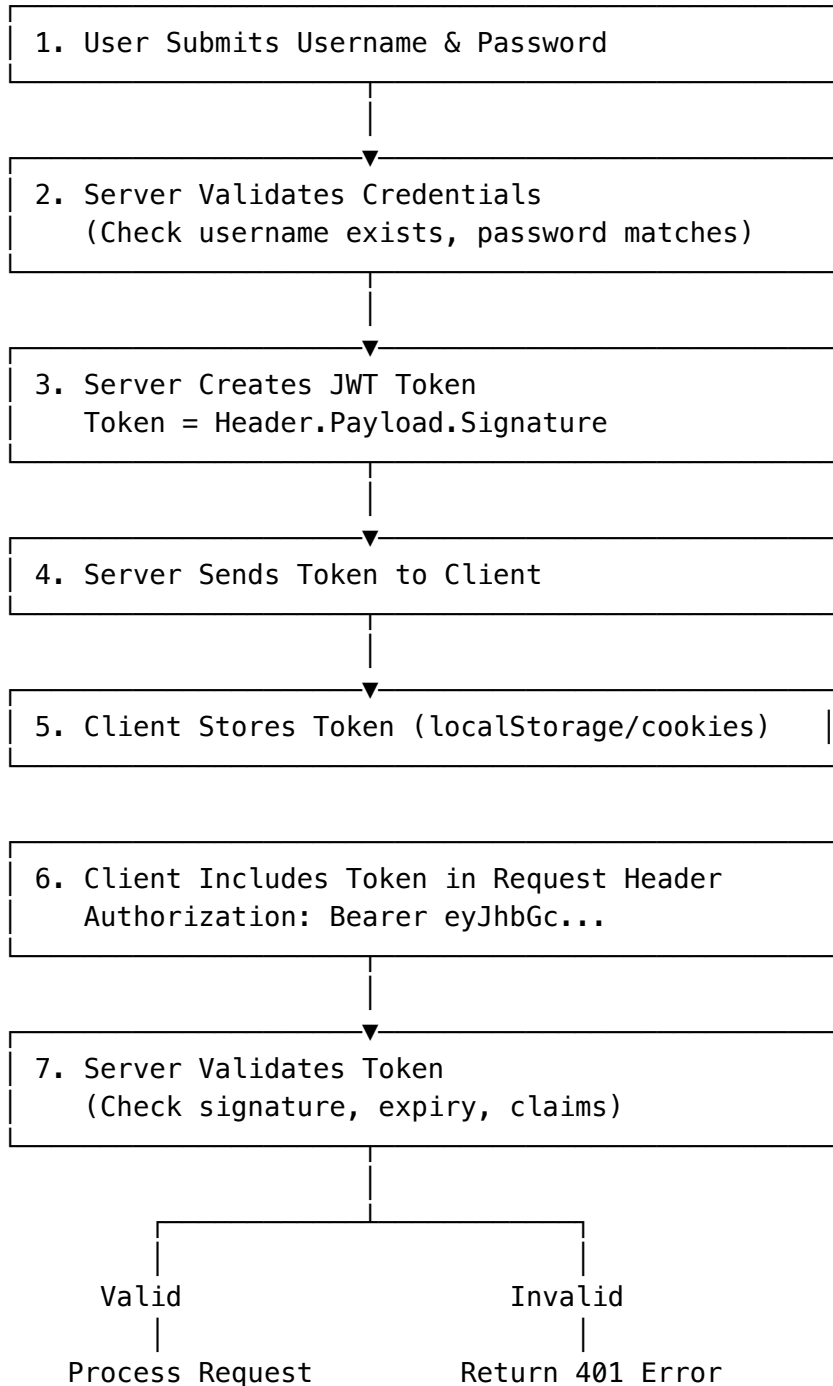
```
{  
  "detail": "Error message here"  
}
```

HTTP Status Codes

- 200 OK: Successful GET/POST
- 201 Created: Resource created
- 204 No Content: Successful DELETE
- 400 Bad Request: Invalid input
- 401 Unauthorized: Missing/invalid token
- 403 Forbidden: Access denied
- 404 Not Found: Resource not found
- 500 Internal Server Error: Server error

Authentication Guide

JWT Token Flow



Registration Example

```
curl -X POST "http://localhost:8000/auth/register" \
  -H "Content-Type: application/json" \
  -d '{
    "username": "john_doe",
    "email": "john@example.com",
```

```
"password": "secure_password_123"
}'
```

Response:

```
{
  "id": 1,
  "username": "john_doe",
  "email": "john@example.com",
  "created_at": "2024-02-04T10:30:00"
}
```

Login Example

```
curl -X POST "http://localhost:8000/auth/login" \
  -H "Content-Type: application/json" \
  -d '{
    "username": "john_doe",
    "password": "secure_password_123"
  }'
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 1800
}
```

Using Token

```
curl -X GET "http://localhost:8000/posts/" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Common Use Cases**Use Case 1: User Registration & Login**

Step 1: Register

POST /auth/register

```
{
  "username": "alice",
  "email": "alice@example.com",
  "password": "password123"
}
```

Step 2: Login

POST /auth/login

```
{  
  "username": "alice",  
  "password": "password123"  
}
```

Response includes: access_token

Use Case 2: Create & Publish Post

Create post

POST /posts/

Authorization: Bearer <token>

```
{  
  "title": "My First Post",  
  "content": "This is my first blog post on MiniSocial!"  
}
```

Response: Created post with ID

```
{  
  "id": 1,  
  "title": "My First Post",  
  "content": "This is my first blog post on MiniSocial!",  
  "user_id": 1,  
  "created_at": "2024-02-04T10:35:00"  
}
```

Use Case 3: Like a Post

Like post

POST /likes/1

Authorization: Bearer <token>

Unlike post

DELETE /likes/1

Authorization: Bearer <token>

Use Case 4: Comment on Post

Add comment

POST /comments/

Authorization: Bearer <token>

```
{  
  "post_id": 1,  
  "content": "Great post! I enjoyed reading this."  
}
```

Get all comments for post

GET /comments/1

Use Case 5: Follow User & View Feed

Follow user

POST /follow/2

Authorization: Bearer <token>

Get your followers

GET /follow/followers/1

Get posts from followed users

GET /posts/

(Shows posts from users you follow)

Use Case 6: Search

Search posts by keyword

GET /search/posts?q=python

Search users by username

GET /search/users?q=john

Project Structure

Basic Blog App/

```

|
|— app/
|   |— __init__.py           # Package initialization
|   |— main.py              # FastAPI app entry point
|   |— alembic.ini          # Alembic configuration
|   |
|   |— api/
|       |— __init__.py
|       |— deps.py          # Dependency injection (get_db,
|                               get_current_user)
|       |
|       |— routers/
|           |— __init__.py
|           |— auth.py       # Authentication routes
|           |— user.py       # User management routes
|           |— post.py       # Post routes
|           |— comment.py    # Comment routes
|           |— like.py       # Like routes
|           |— follow.py     # Follow routes
|           |— search.py     # Search routes
|       |
|       |— core/
|           |— __init__.py
|           |— config.py     # Settings & configuration
|           |— security.py   # JWT & password utilities

```


— database/	
— __init__.py	
— database.py	# SQLAlchemy setup, session management
— models/	
— __init__.py	
— user.py	# User ORM model
— post.py	# Post ORM model
— comment.py	# Comment ORM model
— like.py	# Like ORM model
— follow.py	# Follow ORM model
— schemas/	
— __init__.py	
— auth.py	# Authentication schemas (register, login)
— user.py	# User request/response schemas
— post.py	# Post schemas
— comment.py	# Comment schemas
— like.py	# Like schemas
— follow.py	# Follow schemas
— search.py	# Search schemas
— services/	
— __init__.py	
— user_service.py	# User business logic
— post_service.py	# Post business logic
— comment_service.py	# Comment business logic
— search_service.py	# Search business logic
— views/	
— base.html	# Base template
— login.html	# Login page
— register.html	# Registration page
— feed.html	# Posts feed page
— profile.html	# User profile page
— post_detail.html	# Single post page
— static/	
— css/	
— style.css	
— js/	
— main.js	
— migrations/	
— env.py	# Alembic migration environment
— script.py.mako	# Migration template
— versions/	# Migration files
— requirements.txt	# Project dependencies
— .env	# Environment variables (DO NOT COMMIT)
— .gitignore	# Git ignore rules

```
|— README.md
|— DOCUMENTATION.md
```

```
# Quick start guide
# This file
```

Key Files Explained

main.py - Initializes FastAPI application - Mounts static files - Configures Jinja2 templates - Includes API routers

deps.py - `get_db()`: Database session dependency - `get_current_user()`: Current authenticated user dependency

config.py - Database URL configuration - JWT settings - API settings

security.py - `hash_password()`: Hash user passwords - `verify_password()`: Verify password against hash - `create_access_token()`: Generate JWT tokens - `verify_token()`: Validate JWT tokens

Development Guide

Adding a New Feature

Step 1: Create Model (models/new_feature.py)

```
from sqlalchemy import Column, Integer, String, ForeignKey
from app.database.database import Base

class NewFeature(Base):
    __tablename__ = "new_features"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    name = Column(String, nullable=False)
```

Step 2: Create Schema (schemas/new_feature.py)

```
from pydantic import BaseModel
from datetime import datetime

class NewFeatureBase(BaseModel):
    name: str

class NewFeatureCreate(NewFeatureBase):
    pass

class NewFeatureOut(NewFeatureBase):
    id: int
    user_id: int
    created_at: datetime
```

```
class Config:
    from_attributes = True
```

Step 3: Create Service (services/new_feature_service.py)

```
from sqlalchemy.orm import Session
from app.models.new_feature import NewFeature
from app.schemas.new_feature import NewFeatureCreate

def create_new_feature(db: Session, feature: NewFeatureCreate, user_id: int):
    db_feature = NewFeature(**feature.dict(), user_id=user_id)
    db.add(db_feature)
    db.commit()
    db.refresh(db_feature)
    return db_feature
```

Step 4: Create Route (api/routers/new_feature.py)

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from app.api.deps import get_db, get_current_user
from app.schemas.new_feature import NewFeatureCreate, NewFeatureOut
from app.services.new_feature_service import create_new_feature

router = APIRouter(prefix="/features", tags=["Features"])

@router.post("/", response_model=NewFeatureOut)
def create_feature(
    feature: NewFeatureCreate,
    db: Session = Depends(get_db),
    current_user = Depends(get_current_user)
):
    return create_new_feature(db, feature, current_user.id)
```

Step 5: Include Router in main.py

```
from app.api.routers import new_feature

app.include_router(new_feature.router)
```

Step 6: Create Migration

```
alembic revision --autogenerate -m "Add new_features table"
alembic upgrade head
```

Testing

```
# Run tests
pytest
```

```
# Run with coverage
pytest --cov=app

# Run specific test file
pytest tests/test_auth.py

# Run with verbose output
pytest -v
```

Code Style

```
# Format code with Black
black app/

# Check with flake8
flake8 app/

# Type checking with mypy
mypy app/
```

Deployment

Option 1: Deploy to Heroku

```
# 1. Install Heroku CLI
brew tap heroku/brew && brew install heroku

# 2. Login
heroku login

# 3. Create app
heroku create your-app-name

# 4. Add PostgreSQL
heroku addons:create heroku-postgresql:hobby-dev

# 5. Set environment variables
heroku config:set SECRET_KEY=your-secret-key
heroku config:set ALGORITHM=HS256

# 6. Deploy
git push heroku main
```

Option 2: Deploy to AWS EC2

1. SSH into EC2 instance

```
ssh -i your-key.pem ec2-user@your-instance-ip
```

2. Install dependencies

```
sudo yum install python3 postgresql postgresql-server git
```

3. Clone repository

```
git clone your-repo-url
```

```
cd Basic-Blog-App
```

4. Create virtual environment and install packages

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install -r requirements.txt
```

5. Create .env file with production settings

```
nano .env
```

6. Run migrations

```
cd app
```

```
alembic upgrade head
```

7. Start with Gunicorn

```
pip install gunicorn
```

```
gunicorn -w 4 -b 0.0.0.0:8000 main:app
```

Option 3: Docker Deployment

Create Dockerfile:

```
FROM python:3.9
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build and run:

```
docker build -t minisocial .
```

```
docker run -p 8000:8000 --env-file .env minisocial
```

Production Checklist

- ☐ Set DEBUG = False in production

- ☐ Generate strong SECRET_KEY
 - ☐ Use HTTPS
 - ☐ Set up proper database backups
 - ☐ Configure CORS settings
 - ☐ Set up logging and monitoring
 - ☐ Use environment variables for sensitive data
 - ☐ Set up rate limiting
 - ☐ Enable HTTPS redirects
 - ☐ Configure allowed hosts
-

Troubleshooting

Problem: Connection refused (psycopg2.OperationalError)

Cause: PostgreSQL is not running or connection string is wrong

Solution:

```
# Start PostgreSQL
brew services start postgresql # macOS
sudo service postgresql start  # Linux
pg_ctl start                    # Windows

# Verify connection
psql -U postgres -d blog_db

# Check connection string in .env
DATABASE_URL=postgresql://username:password@localhost:5432/blog_db
```

Problem: “No such file or directory: alembic.ini”

Cause: Running command from wrong directory

Solution:

```
cd app # Navigate to app directory first
alembic upgrade head
```

Problem: JWT token expired

Cause: Token expires after configured time (default 30 minutes)

Solution:

```
# Login again to get new token
# OR increase ACCESS_TOKEN_EXPIRE_MINUTES in .env
ACCESS_TOKEN_EXPIRE_MINUTES=120 # 2 hours
```

Problem: “UNIQUE constraint failed”

Cause: Duplicate entry (e.g., duplicate username)

Solution:

```
# Use unique check before insert
existing_user = db.query(User).filter(User.username == username).first()
if existing_user:
    raise ValueError("Username already exists")
```

Problem: Migrations not being detected

Cause: alembic.ini or env.py configuration issue

Solution:

```
# Ensure alembic is initialized
alembic current # Check current revision

# Force migration detection
alembic revision --autogenerate -m "migration name"
```

Problem: “ModuleNotFoundError: No module named ‘app’”

Cause: Python path not set correctly or not in correct directory

Solution:

```
# Run from app directory
cd app
uvicorn main:app --reload

# OR set PYTHONPATH
export PYTHONPATH="${PYTHONPATH}:${pwd}"
```

Contributing

Development Workflow

1. Fork the repository

```
git clone https://github.com/devberatzengin/Basic-Blog-App.git
```

2. Create feature branch

```
git checkout -b feature/amazing-feature
```

3. Commit changes

```
git commit -m "Add amazing feature"  
git commit -m "Fix bug in authentication"
```

4. Push to branch

```
git push origin feature/amazing-feature
```

5. Open Pull Request

- Describe your changes clearly
- Reference any related issues
- Ensure tests pass

Code Style Guidelines

- Follow PEP 8
- Use meaningful variable names
- Add docstrings to functions
- Keep functions small and focused
- Add comments for complex logic
- Use type hints

Commit Message Format

[TYPE] Short description (50 chars max)

Detailed explanation of changes (if needed)

- Point 1
- Point 2

Fixes #123

Types: feat, fix, docs, style, refactor, test, chore

FAQ

Q: How do I reset the database? A:

```
dropdb blog_db  
createdb blog_db  
cd app  
alembic upgrade head
```

Q: How do I change password? A: There's no dedicated endpoint yet. Users need to update password via database or implement password change endpoint.

Q: Can I use SQLite instead of PostgreSQL? A: Yes, but it's not recommended for production. Update DATABASE_URL to sqlite:///./blog.db

Q: How do I add role-based access control? A: Add a role column to Users table and check role in route dependencies.

Q: Is this production-ready? A: Not yet. Add: rate limiting, input validation, CORS configuration, error logging.

License

This project is open source. See LICENSE file for details.

Support

- **Issues:** [GitHub Issues](#)
 - **Discussions:** [GitHub Discussions](#)
 - **Email:** Contact project maintainer
-

Version History

Version	Date	Changes
1.0.0	Feb 2024	Initial release
0.9.0	Jan 2024	Beta release

Last Updated: February 4, 2026
Maintainer: [@devberatzengin](#)